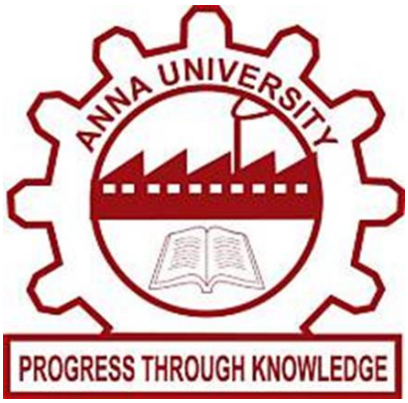
INTRUSION DETECTION SYSTEM

A SUMMER INTERNSHIP REPORT

Submitted by

AJAY A – 810023205043



DEPARTMENT OF INFORMATION TECHNOLOGY

ANNA UNIVERSITY,

TIRUCHIRAPPALLI – 620 024.

UNIVERSITY COLLEGE OF ENGINEERING (BIT CAMPUS),

ANNA UNIVERSITY, TIRUCHIRAPPALLI – 620 024

DEPARTMENT OF INFORMATION TECHNOLOGY

BONAFIDE CERTIFICATE

This is to certify that this Summer Internship project report **"Intrusion Detection System"** is the Bonafide work of **AJAY A (810023205043)** who has satisfactorily completed the course of **IT3711 – SUMMER INTERNSHIP** for **SEVENTH** semester, **B.TECH INFORMATION TECHNOLOGY** during the academic year **2025-2026**.

Faculty Incharge **Head of the Department**

Submitted for the practical examination held on _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Fantasy Solution, Tiruchirappalli**, for providing me with the opportunity to undertake this internship in the domain of Cyber Security. This experience has been immensely valuable in strengthening my practical understanding of network security and intrusion detection.

I am deeply thankful to the management, the Project Trainer and the staff of Fantasy Solution for their guidance, technical support and encouragement throughout the duration of this project. Their support helped me to understand the various stages of the project, from setting up the laboratory to testing and analysing the system.

I also extend my heartfelt thanks to the Class Coordinator and the faculty of the Department of Information Technology, University College of Engineering (BIT Campus), Anna University, Tiruchirappalli, for their continuous support and for facilitating this internship opportunity as part of the academic curriculum.

Finally, I am grateful to my family and friends for their support and motivation, which helped me complete this project successfully within the stipulated duration.

AJAY A

ABSTRACT

The Intrusion Detection System is a network security project developed to monitor network traffic, detect suspicious activity and report it to the administrator in a clear form. Firewalls and antivirus software alone cannot detect attacks that use allowed ports, such as repeated password guessing on an SSH service or malicious requests to a web application. The project was undertaken during a Cyber Security internship at Fantasy Solution, Tiruchirappalli, and provides a low-cost, open-source detection solution built and tested in an isolated virtual laboratory.

The project is implemented using Snort as the rule-based detection engine and Python with the Scapy library for a threshold-based detector and a log analysis script. Custom rules were written to detect ICMP ping sweeps, TCP SYN scans, SSH brute-force attempts and SQL injection attempts. The Python detector identifies port scanning by counting the distinct ports contacted by a source within a time window. Wireshark and tcpdump were used to verify the traffic, and Nmap and curl were used from a Kali Linux machine to generate controlled test traffic.

One of the main objectives of the system is to give early warning of reconnaissance and credential attacks, to reduce manual log checking, and to help the administrator find when, from where and against which service an attack was attempted. By combining signature-based and threshold-based detection, the system provides better coverage than either method alone, and tuning of the rules reduces false positives.

Overall, the Intrusion Detection System demonstrates the practical application of network monitoring, attack analysis and scripting. It highlights how simple open-source tools can provide strong visibility of a network, and it can be extended in the future with machine learning, SIEM integration and automatic response.

TABLE OF CONTENTS

CHAPTER / TITLE **PAGE NO**

1. Introduction **1**
1.1 OBJECTIVES OF THE INTERNSHIP 2
1.2 MOTIVATION 2
2. Company Profile **3**
About the Company: 3
Company Details: 3
Tools and Technologies Used 3
3. Daily Work Report **4**
4. Block Diagram **6**
5. Intrusion Detection System **6**
5.1 Existing System 6
5.2 Proposed System 7
5.3 Capabilities 8
5.4 System Requirements 8
5.5 Types and Detection Methods of Intrusion Detection Systems 8
6. Proposed Method **9**
6.1 Packet Capture 9
6.2 Packet Decoding and Preprocessing 9
6.3 Signature-Based Detection 10
6.4 Anomaly (Threshold) Detection 10
6.5 Alert Generation and Logging 10
6.6 Alert Analysis and Response 11
6.7 Rule Management and Tuning 11
6.8 Evaluation Metrics 11
7. Algorithm **11**
7.1 Packet Capture and Preprocessing – Algorithm 11
7.2 Signature Matching – Algorithm 12
7.3 Port Scan Detection (Python) – Algorithm 12
7.4 SSH Brute-Force Detection – Algorithm 12
7.5 Alert Generation and Logging – Algorithm 12
7.6 Alert Analysis – Algorithm 13
8. Code Implementation **14**
8.1 Snort Configuration Code 14
8.2 Custom Detection Rules Code 14
8.3 Python Port Scan Detector Code 14
8.4 Log Summary Script Code 15
8.5 Commands Used to Run and Test the System 15
9. Sample Output **16**
9.1 Configuration Validation 16
9.2 Snort Console Alerts 16

TABLE OF CONTENTS

CHAPTER / TITLE **PAGE NO**

9.3 Python Port Scan Detector Output 16
9.4 Alert Summary Output 17
10. Analysis **17**
Test Cases 18
10.1 Challenges Faced 18
11. Conclusion **18**
Learning Outcomes 19
12. Future Work **19**
13. References **20**

1. INTRODUCTION

The **Intrusion Detection System (IDS)** is a network security project that monitors network traffic, compares it with known attack patterns and normal behaviour, and raises an alert whenever suspicious activity is observed. In today's connected world, every organisation depends on networks and web services for communication, payments and the storage of important data. At the same time, attackers use automated tools that can scan thousands of systems within minutes, guess passwords repeatedly, and send malicious requests to web applications. Preventive controls such as firewalls and antivirus software are necessary, but no preventive control is perfect, and an intrusion that is not noticed can continue for days. A system that can *detect* an attack in progress is therefore an essential second layer of defence.

The primary purpose of this project is to study, design and demonstrate a low-cost, open-source intrusion detection system in a controlled laboratory environment. The system captures network packets, inspects them using signature-based rules written for **Snort**, and also applies a threshold-based behavioural check written in **Python** to find activities such as port scanning, SSH password guessing, ICMP ping sweeps and SQL injection attempts. The alerts are written to log files and are analysed with a small log summary script, so that the administrator can understand what happened, from which address, and against which service.

The project was developed using the following technologies. **Snort** is the rule-based detection engine. **Python 3 with the Scapy library** is used to build a custom anomaly detector and the log analysis script. **Wireshark** and **tcpdump** are used to capture and verify traffic. **Nmap** and **curl** are used from a **Kali Linux** machine to generate controlled test traffic. The sensor and the target run on **Ubuntu Server** virtual machines created with **VirtualBox** on a host-only network, so that none of the test traffic can reach any real network.

The system follows a simple three-stage architecture consisting of the **Traffic Layer** (hosts and a switch that mirrors traffic to the sensor), the **Detection Layer** (packet capture, decoding, Snort rules and the Python detector) and the **Analysis Layer** (alert logs, log summary script and the analyst who responds to the alerts). Packets flow from the network to the sensor, the sensor decides whether they are suspicious, and the result is stored and reported to the analyst.

The IDS consists of several modules, including **Packet Capture, Packet Decoding and Preprocessing, Signature-Based Detection, Anomaly (Threshold) Detection, Alert Generation and Logging, and Alert Analysis**. Each module performs a specific function and works together to provide a complete detection solution. Overall, the project provides practical knowledge of network protocols, attack techniques, detection rule writing, scripting and incident analysis, and it can be extended in the future with features such as **machine learning based detection, SIEM integration, a web dashboard and automatic response**.

1.1 OBJECTIVES OF THE INTERNSHIP

Specific Objectives

The main objective of the Intrusion Detection System project is to understand how network attacks appear in traffic and to build a working system that detects them and reports them clearly. The objectives of the project are:

- To understand the working principles, components and classification of intrusion detection systems.
- To set up an isolated virtual laboratory with a sensor machine, a target machine and a test machine.
- To install, configure and validate **Snort** as a network-based intrusion detection engine.
- To write **custom detection rules** for ICMP ping sweeps, TCP SYN scans, SSH brute-force attempts and SQL injection attempts.
- To develop a **Python-based detector** that identifies port scanning using a time window and a threshold.
- To generate controlled test traffic and observe the alerts produced by the system.
- To analyse the alerts, classify them as true or false positives and tune the rules.
- To evaluate the system using standard metrics such as precision, recall and false positive rate.
- To follow safe and ethical testing practices by performing all tests only in an isolated laboratory.

1.2 MOTIVATION

The motivation behind developing the Intrusion Detection System is to make network attacks visible. Many small organisations depend only on a firewall and an antivirus program. A firewall allows traffic to public services such as web and SSH, and the antivirus only checks files on the computer, so attacks that use allowed ports, such as repeated password guessing on an open SSH service or malicious requests sent to a website, are not noticed by either of them.

This project provides a practical way to **detect reconnaissance, credential attacks and web attacks at an early stage**. Detecting an attacker during scanning or password guessing gives the defender time to respond before data is stolen or systems are damaged. The project also shows how simple and inexpensive tools can provide a strong monitoring capability.

Another major motivation is to gain practical exposure to cyber security tools and the thinking of a security analyst. Working with **Snort, Wireshark, Nmap, Python and Linux** shows how theory about protocols and attacks is applied in real monitoring work, and it builds skills that are useful for careers in cyber security and network administration.

2. COMPANY PROFILE

About the Company:

Fantasy Solution is an organisation located in Tiruchirappalli (Trichy), Tamil Nadu, India, at #16, Sannath Plaza, 3rd Floor, Melapudur. The organisation provides industrial training and internship opportunities and project guidance to engineering students, helping them bridge the gap between academic knowledge and practical industry requirements. During the internship, students work on a project under the guidance of a Project Trainer, who acts as the industrial expert and joint supervisor.

As part of my summer internship at Fantasy Solution, I worked in the **Cyber Security** domain and developed an **Intrusion Detection System**. The project involved setting up a virtual laboratory, installing and configuring Snort, writing detection rules, developing a Python-based detector for port scanning, generating controlled test traffic and analysing the alerts. The internship helped me understand how network attacks appear in traffic and how they can be detected and reported.

Overall, the internship at Fantasy Solution provided an excellent opportunity to apply theoretical concepts of computer networks and information security in a practical environment. The experience strengthened my technical skills in Linux, network monitoring, Snort rule writing and Python scripting.

Company Details:

- **Company Name:** Fantasy Solution
- **Address:** #16, Sannath Plaza, 3rd Floor, Melapudur, Tiruchirappalli, Tamil Nadu, India
- **Telephone:** 0431-4250535
- **Internship Domain:** Cyber Security
- **Internship Duration:** 01/06/2026 to 30/06/2026
- **Project Completed During Internship:** Intrusion Detection System

Tools and Technologies Used

The following tools and technologies were used to build, run and test the project in the virtual laboratory.

Category	Tool / Technology
Operating Systems	Ubuntu Server (sensor and target), Kali Linux (test machine)
Virtualisation	Oracle VirtualBox with a host-only network
Detection Engine	Snort (signature-based rules)
Programming Language	Python 3
Libraries	Scapy, collections, re, time

Category	Tool / Technology
Packet Analysis	Wireshark, tcpdump
Test Traffic Tools	Nmap, curl, ping, ssh client
Target Services	Apache web server, OpenSSH server
Development Tools	Terminal, Nano / Visual Studio Code

3. DAILY WORK REPORT

Day 1: Attended the orientation session and was briefed on the internship guidelines, evaluation criteria and reporting format. Discussed and finalized the project topic, “Intrusion Detection System”.

Day 2: Conducted requirement gathering. Listed the stakeholders (Network Administrator and Security Analyst) and identified the core functional requirements of the detection system.

Day 3: Carried out a study of existing network protection methods such as firewalls and antivirus software and documented their limitations in detecting attacks on allowed ports.

Day 4: Finalized the scope and objectives of the project. Prepared the list of hardware and software requirements needed for development.

Day 5: Set up the development environment: installed VirtualBox, created Ubuntu Server and Kali Linux virtual machines and updated the packages.

Day 6: Revised networking fundamentals: the TCP/IP model, the three-way handshake, TCP flags, ICMP and common ports used by services such as SSH and HTTP.

Day 7: Studied the classification of intrusion detection systems (NIDS, HIDS, hybrid) and the detection methods (signature, anomaly, stateful protocol and hybrid).

Day 8: Designed the laboratory topology with a sensor, a target server and a test machine on a host-only network, and enabled promiscuous mode on the sensor interface.

Day 9: Installed Snort on the sensor machine and studied its directory structure, configuration file and rule files.

Day 10: Configured the HOME_NET and EXTERNAL_NET variables in snort.conf and validated the configuration using the test option (-T).

Day 11: Studied the Snort rule syntax: rule header, direction operator, and rule options such as msg, flags, content, itype and sid.

Day 12: Wrote the first custom rule to detect ICMP echo requests and verified it with a ping from the test machine.

Day 13: Captured normal traffic with Wireshark and tcpdump to understand the baseline behaviour of the network.

Day 14: Wrote the TCP SYN scan detection rule using the detection_filter option and tested it with an Nmap scan of the target in the laboratory.

Day 15: Wrote the SSH brute-force detection rule with a count and time window, and tested it with repeated connection attempts.

Day 16: Installed the Apache web server on the target and wrote the SQL injection detection rule that looks for the keywords UNION and SELECT in web requests.

Day 17: Studied the Snort alert formats and the location of the alert log files, and practised reading alerts.

Day 18: Studied the Scapy library: sniffing packets, accessing IP and TCP layers and reading TCP flags.

Day 19: Developed the Python port scan detector that reads TCP SYN packets and records the destination ports contacted by each source address.

Day 20: Added the sliding time window and the threshold logic to the detector and tested it against Nmap scans.

Day 21: Developed the log summary script that reads the Snort alert file and counts the alerts by rule and by source address.

Day 22: Ran Snort and the Python detector together and verified that both generate alerts for the same scan from different detection approaches.

Day 23: Executed all planned test scenarios (ping, SYN scan, SSH attempts and web requests) and recorded the alerts generated.

Day 24: Tested normal activities such as web browsing and a single SSH login to check for false positives.

Day 25: Tuned the thresholds and time windows of the rules to reduce false positives while keeping the detection of attacks.

Day 26: Studied the evaluation metrics (true positive, false positive, precision, recall) and mapped the detected attacks to the MITRE ATT&CK framework.

Day 27: Studied the limitations of the system such as encrypted traffic, evasion techniques and the need for regular rule updates, and documented the future improvements.

Day 28: Reviewed and cleaned the code and the rule file, and verified that the complete system works from a fresh start.

Day 29: Captured the sample outputs of the working system and prepared the output section of the report.

Day 30: Completed the documentation, compiled the final internship report and prepared for the final presentation.

4. BLOCK DIAGRAM

The block diagram shows how the network traffic, the IDS sensor and the analyst interact with each other. Normal users and a test attacker send traffic to the target server. A switch mirrors a copy of this traffic to the sensor, which captures the packets, decodes and normalises them, and checks them against the Snort rules and the Python threshold logic. If a rule matches or a threshold is crossed, an alert is generated and written to the log. The analyst then reviews the alert, confirms whether it is a real attack or a false positive, and decides the action.

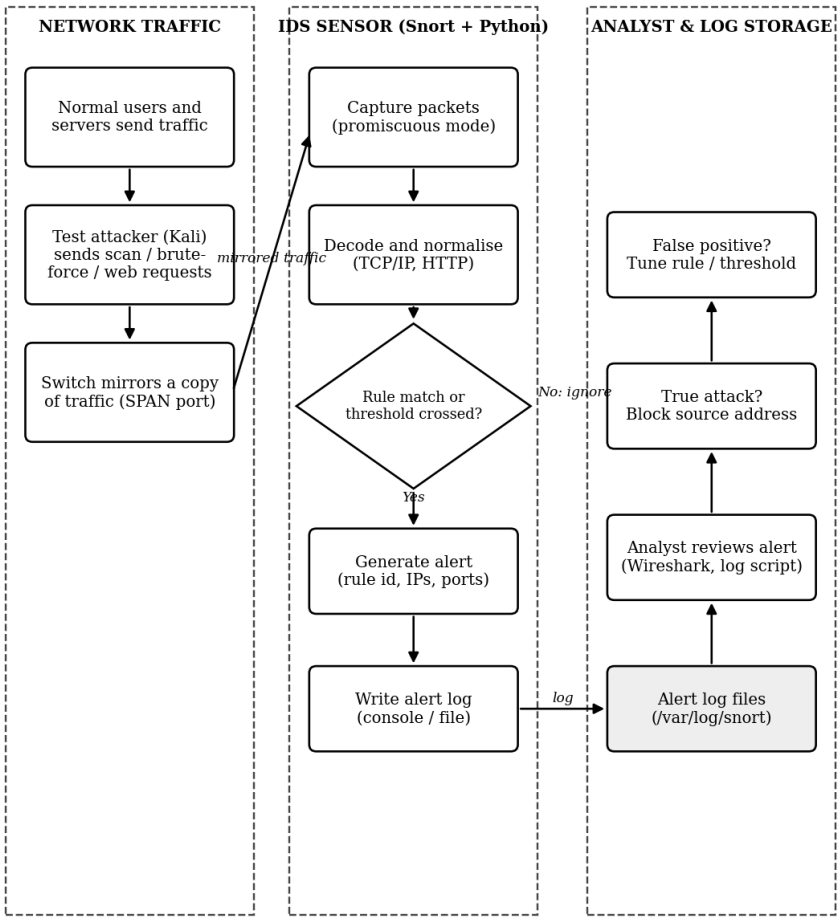


Figure 4.1: Block diagram of the Intrusion Detection System

5. INTRUSION DETECTION SYSTEM

5.1 Existing System

Most small organisations and training centres protect their networks with a perimeter firewall and an antivirus program. The firewall allows or blocks traffic according to address, port and protocol, and the antivirus inspects files on the computers. There is little or no monitoring of what actually happens inside the allowed traffic. This existing approach suffers from several limitations:

- **No visibility into allowed traffic** – attacks that use permitted ports, such as SSH or HTTP, pass through the firewall unnoticed.
- **Late discovery of intrusions** – an attacker may continue scanning or guessing passwords for days before anyone notices.
- **Scattered logs** – firewall, server and application logs are stored separately and are rarely reviewed together.
- **Manual monitoring** – checking logs by hand is slow, tiring and error-prone.
- **No early warning** – reconnaissance activity such as port scanning gives no warning to the administrator.
- **Limited detection of web attacks** – malicious requests such as SQL injection look like normal web traffic to a firewall.
- **Difficulty in incident analysis** – without recorded alerts it is hard to find when, from where and how an attack happened.

These drawbacks make the existing approach less effective against modern attacks. Therefore, there is a need for a monitoring system that can automatically detect suspicious activity and report it clearly.

5.2 Proposed System

The proposed **Intrusion Detection System** is a network-based monitoring solution that passively observes a copy of the network traffic, detects known attack patterns and abnormal behaviour, and records alerts for the administrator. It is developed using **Snort** for rule-based detection and **Python with Scapy** for threshold-based behavioural detection. Key aspects of the proposed system include:

- **Passive monitoring** – the sensor works on a mirrored copy of the traffic, so it does not slow down the network.
- **Signature-based detection** – custom Snort rules detect ping sweeps, SYN scans, SSH brute-force attempts and SQL injection keywords.
- **Anomaly (threshold) detection** – the Python detector counts the distinct ports contacted by a source within a time window and raises an alert when a threshold is crossed.
- **Hybrid approach** – signatures handle known patterns and thresholds handle suspicious behaviour, giving better coverage.
- **Clear alerts** – each alert shows the time, rule identifier, message, protocol, source and destination.
- **Log analysis** – a summary script counts alerts by rule and by source address.
- **Safe laboratory design** – all testing is done on an isolated host-only virtual network.
- **Low cost** – only open-source software is used.

5.3 Capabilities

- **Packet Capture** – Captures packets from the network interface in promiscuous mode.
- **Packet Decoding** – Decodes Ethernet, IP, TCP, UDP and ICMP headers and normalises HTTP data.
- **ICMP Detection** – Detects ICMP echo requests used in ping sweeps.
- **Scan Detection** – Detects TCP SYN scans using a rate-based rule and the Python detector.
- **Brute-Force Detection** – Detects repeated connection attempts to the SSH service.
- **Web Attack Detection** – Detects SQL injection keywords in HTTP requests.
- **Alert Logging** – Writes alerts to the console and to log files.
- **Alert Summary** – Summarises the alerts by rule and source address.

5.4 System Requirements

Requirement	Specification
Operating System	Ubuntu Server (sensor and target), Kali Linux (test machine)
Programming Language	Python 3
Detection Engine	Snort
Libraries	Scapy
Packet Analysis Tools	Wireshark, tcpdump
Virtualisation	VirtualBox with host-only network and promiscuous mode enabled
Hardware (Development)	Intel Core i3 or higher, 8 GB RAM, 40 GB free disk space
Network	Isolated host-only virtual network

5.5 Types and Detection Methods of Intrusion Detection Systems

Intrusion detection systems are classified according to where they are placed and how they decide that an event is malicious. A network-based IDS (NIDS) inspects packets on a network segment and can protect many hosts with a single sensor, whereas a host-based IDS (HIDS) is installed on one computer and examines logs, files and processes. A hybrid IDS combines both. The system developed in this project is a **network-based IDS**.

Method	How it works	Strengths	Weaknesses
Signature-based	Compares traffic with patterns of known attacks	Accurate for known attacks, few false positives	Cannot detect new attacks
Anomaly-based	Compares behaviour with a normal profile or threshold	Can detect unknown activity	More false positives, needs tuning
Stateful protocol analysis	Checks protocol behaviour against standards	Finds protocol misuse	Needs more resources

Method	How it works	Strengths	Weaknesses
Hybrid	Combines the above methods	Balanced coverage	More complex to maintain

6. PROPOSED METHOD

The project follows a structured pipeline in which packets travel from the network to the analyst:

Packet Capture → Decoding and Preprocessing → Detection (Rules and Threshold) → Alert Generation → Logging → Analysis and Response

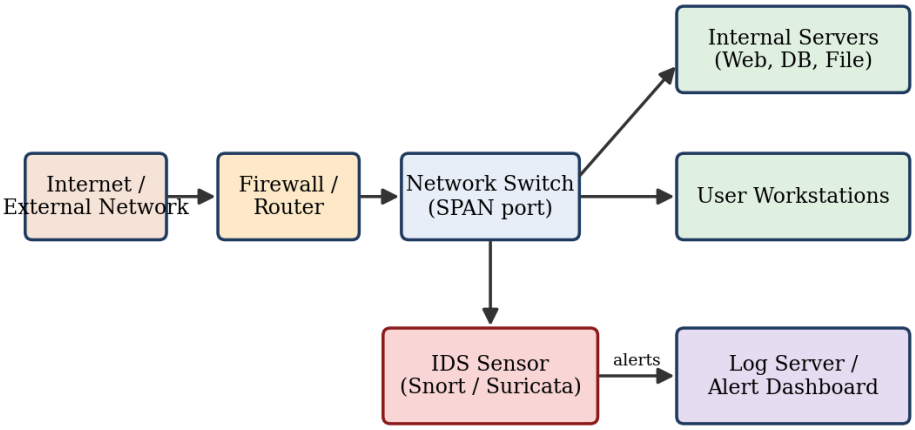


Figure 6.1: Deployment of the IDS sensor in the test network

6.1 Packet Capture

- The sensor interface is placed in promiscuous mode so that packets not addressed to the sensor are also received.
- The switch (or virtual network) mirrors the traffic of the target to the sensor, so the sensor works on a copy of the traffic.
- The libpcap library is used by Snort and Scapy to read the packets from the interface.
- Normal traffic is captured first with Wireshark to understand the baseline behaviour.

6.2 Packet Decoding and Preprocessing

- Decodes the Ethernet, IP, TCP, UDP and ICMP headers of every packet.
- Reassembles fragmented packets and TCP streams so that an attack split into small pieces can still be detected.
- Normalises HTTP requests so that encoding tricks do not hide keywords from the rules.
- Passes the processed packet to the detection engine.

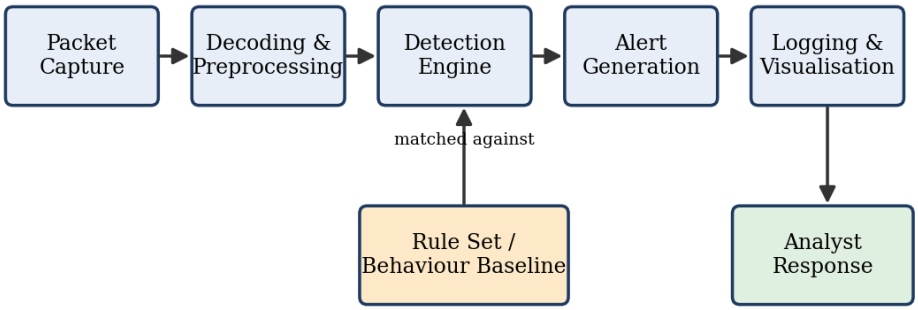


Figure 6.2: Packet processing pipeline of the IDS

6.3 Signature-Based Detection

- Snort compares every packet with the rules in the local rules file.
- A rule has a header (action, protocol, addresses, ports, direction) and options (message, flags, content, thresholds, sid).
- The ICMP rule matches echo requests; the content rule looks for UNION followed by SELECT in web requests.
- The detection_filter option raises an alert only after a specified number of matching packets from one source in a given time.
- Local rules use identifiers above 1,000,000 so that they do not conflict with official rules.

6.4 Anomaly (Threshold) Detection

- The Python detector reads TCP SYN packets using Scapy.
- For every source address it stores the destination ports contacted within a sliding time window (10 seconds).
- If the number of distinct ports reaches the threshold (15 ports), a port scan alert is printed.
- After an alert the history of that source is cleared to avoid repeated alerts for the same scan.
- The window and threshold can be changed to suit the normal traffic of the network.

6.5 Alert Generation and Logging

- Each alert contains the time, rule identifier, message, priority, protocol, source and destination.
- Alerts are shown on the console during development and written to the alert log file.
- The Python detector prints alerts with the time and the source address.

- The log files are stored on the sensor and can be forwarded to a central log server in the future.

6.6 Alert Analysis and Response

- The analyst reads the rule message and rule identifier to understand what was detected.
- The source and destination addresses and ports are checked to identify the systems involved.
- The related packets are opened in Wireshark to confirm the content.
- The alert is classified as a true positive or a false positive.
- For a true positive the source address is blocked on the firewall and the incident is noted; for a false positive the rule threshold is adjusted.

6.7 Rule Management and Tuning

- Rules are written in a single local.rules file so that they are easy to review.
- Thresholds are tuned after testing with both attack and normal traffic.
- The rule set is validated with the Snort test option before every run.
- Rules are reviewed regularly and updated when the network changes.

6.8 Evaluation Metrics

The detection quality is measured by comparing the decisions of the system with the real nature of the traffic. A true positive (TP) is an attack that was detected, a false positive (FP) is normal activity reported as an attack, a true negative (TN) is normal activity that was ignored, and a false negative (FN) is an attack that was missed.

Metric	Formula	Meaning
Precision	TP / (TP + FP)	How many of the alerts are real attacks
Recall (Detection rate)	TP / (TP + FN)	How many of the real attacks were detected
False positive rate	FP / (FP + TN)	How often normal traffic raises an alert
Accuracy	(TP + TN) / (TP + TN + FP + FN)	Overall proportion of correct decisions

7. ALGORITHM

7.1 Packet Capture and Preprocessing – Algorithm

1. Start the sensor and put the network interface in promiscuous mode.
2. Capture a packet from the interface.
3. Decode the Ethernet, IP and transport layer headers.

4. Reassemble fragments and TCP streams if required.
5. Normalise application data such as HTTP requests.
6. Pass the packet to the detection engine.

7.2 Signature Matching – Algorithm

1. Load the rules from the local rules file and validate them.
2. For every packet, check the protocol, addresses, ports and direction in the rule header.
3. If the header matches, check the rule options such as flags, content and itype.
4. If all options match, mark the rule as matched.
5. If the rule uses detection_filter, count the match for the source address within the time window.
6. Raise an alert only when the count reaches the configured limit.

7.3 Port Scan Detection (Python) – Algorithm

1. Start sniffing TCP packets on the interface.
2. For every packet, check that the SYN flag alone is set.
3. Read the source address, the destination port and the current time.
4. Add the (time, port) pair to the history of that source address.
5. Remove entries older than the time window (10 seconds).
6. Count the distinct ports in the remaining history.
7. If the count is equal to or greater than the threshold (15), print a port scan alert.
8. Clear the history of that source address and continue.

7.4 SSH Brute-Force Detection – Algorithm

1. Read each TCP packet sent to port 22 of the protected network.
2. Check that it is a connection request (SYN flag).
3. Count the requests from each source address within 60 seconds.
4. If the count reaches 5, raise the alert “Possible SSH Brute Force”.
5. Write the alert to the log and continue monitoring.

7.5 Alert Generation and Logging – Algorithm

1. Receive the matched rule or threshold event from the detection module.
2. Collect the time, rule identifier, message, protocol, source and destination.
3. Format the alert in the standard alert format.
4. Display the alert on the console.
5. Append the alert to the alert log file.

7.6 Alert Analysis – Algorithm

1. Open the alert log file and read each alert.
2. Extract the rule identifier, message and source address using a regular expression.
3. Count the alerts for every rule and for every source address.
4. Display the summary table and the list of top source addresses.
5. The analyst checks the packets in Wireshark and classifies the alerts as true or false positives.

8. CODE IMPLEMENTATION

The Intrusion Detection System is implemented using Snort rules and Python scripts. The implementation includes the Snort configuration, the custom detection rules, the Python port scan detector, the log summary script and the commands used to run the system.

8.1 Snort Configuration Code

The protected network is defined and the local rules file is included in the main configuration file. The configuration is then validated before the sensor is started.

```
sudo apt update
sudo apt install snort -y

# /etc/snort/snort.conf  (important lines)
ipvar HOME_NET 192.168.56.0/24
ipvar EXTERNAL_NET !$HOME_NET
include $RULE_PATH/local.rules

# validate the configuration
sudo snort -T -c /etc/snort/snort.conf -i eth0
```

8.2 Custom Detection Rules Code

The rules are written in the local.rules file. Rule 1 detects ICMP echo requests. Rules 2 and 3 use detection_filter to alert only after repeated packets from one source. Rule 4 looks for SQL injection keywords in web requests.

```
# 1. ICMP echo request (ping) detection
alert icmp any any -> $HOME_NET any (msg:"ICMP Ping Detected"; itype:8; \
  sid:1000001; rev:1;)

# 2. Possible SSH brute force: 5 SYN packets in 60 seconds from one source
alert tcp any any -> $HOME_NET 22 (msg:"Possible SSH Brute Force"; flags:S; \
  detection_filter:track by_src, count 5, seconds 60; sid:1000002; rev:1;)

# 3. Possible TCP SYN scan: 20 SYN packets in 5 seconds from one source
alert tcp any any -> $HOME_NET any (msg:"Possible TCP SYN Scan"; flags:S; \
  detection_filter:track by_src, count 20, seconds 5; sid:1000003; rev:1;)

# 4. SQL injection keywords in HTTP traffic
alert tcp any any -> $HOME_NET 80 (msg:"Possible SQL Injection Attempt"; \
  flow:to_server,established; content:"union"; nocase; \
  content:"select"; nocase; distance:0; sid:1000004; rev:1;)
```

8.3 Python Port Scan Detector Code

The detector reads TCP SYN packets with Scapy, keeps a sliding window of the ports contacted by each source address and prints an alert when the number of distinct ports crosses the threshold.

```
from scapy.all import sniff, IP, TCP
from collections import defaultdict
import time

WINDOW = 10        # seconds
THRESHOLD = 15     # distinct ports per window
seen = defaultdict(list)   # source -> [(timestamp, port), ...]
def inspect(pkt):
```

```
    if IP in pkt and TCP in pkt and pkt[TCP].flags == "S":
        src, port, now = pkt[IP].src, pkt[TCP].dport, time.time()
        seen[src].append((now, port))
        seen[src] = [(t, p) for t, p in seen[src] if now - t <= WINDOW]
        ports = {p for _, p in seen[src]}
        if len(ports) >= THRESHOLD:
            stamp = time.strftime("%H:%M:%S")
            print(f"[ALERT] {stamp} Possible port scan from {src}: "
                  f"{len(ports)} ports in {WINDOW}s")
            seen[src].clear()

print(f"[*] Monitoring eth0  window={WINDOW}s  threshold={THRESHOLD} ports")
sniff(iface="eth0", filter="tcp", prn=inspect, store=False)
```

8.4 Log Summary Script Code

The script reads the Snort alert file, extracts the rule identifier, the message and the source address of every alert, and prints the number of alerts for each rule and the top source addresses.

```
import re
from collections import Counter

pattern = re.compile(
    r"\[\*\*\] \[\d+:(\d+):\d+\] (.+?) \[\*\*\].*?\{\w+\} ([\d.]+)")
rules, sources = Counter(), Counter()
names = {}

with open("/var/log/snort/alert") as f:
    for line in f:
        m = pattern.search(line)
        if m:
            sid, msg, src = m.groups()
            rules[sid] += 1
            sources[src] += 1
            names[sid] = msg

print("SID       ALERTS  MESSAGE")
for sid, n in sorted(rules.items()):
    print(f"{sid:<10}{n:>5}  {names[sid]}")
print("\nTop source addresses:")
for src, n in sources.most_common(3):
    print(f"  {src}  ->  {n} alerts")
```

8.5 Commands Used to Run and Test the System

```
# on the sensor: run Snort in console alert mode
sudo snort -A console -q -c /etc/snort/snort.conf -i eth0

# on the sensor: run the Python detector
sudo python3 portscan_detector.py

# on the test machine (Kali): generate controlled test traffic
ping -c 3 192.168.56.20
nmap -sS -p 1-100 192.168.56.20
curl "http://192.168.56.20/?id=1 union select 1,2"
```

9. SAMPLE OUTPUT

The outputs in this chapter illustrate the format of the results produced by the system in the laboratory network (sensor 192.168.56.10, target 192.168.56.20, test machine 192.168.56.30).

9.1 Configuration Validation

Displays the result of the Snort configuration test. The four custom rules are read successfully and Snort reports that the configuration is valid.

```
Terminal - Configuration test
ajay@sensor:~$ sudo snort -T -c /etc/snort/snort.conf -i eth0
Running in Test mode
        --== Initializing Snort ==--
Initializing Output Plugins!
Initializing Preprocessors!
Parsing Rules file "/etc/snort/snort.conf"
Reading rules: local.rules
        4 Snort rules read
        4 detection rules
        0 decoder rules
        0 preprocessor rules
        4 Option Chains linked into 4 Chain Headers
+++++++++++++++++++++++++++++++++++++++++++++++++++
Snort successfully validated the configuration!
Snort exiting
```

Figure 9.1: Snort configuration test

9.2 Snort Console Alerts

Displays the alerts generated by Snort while the test traffic is sent from the test machine. Each alert shows the time, rule identifier, message, protocol, source and destination.

```
Terminal - Snort console alerts
ajay@sensor:~$ sudo snort -A console -q -c /etc/snort/snort.conf -i eth0
06/15-11:20:42.532145 [**] [1:1000001:1] ICMP Ping Detected [**]
   [Priority: 0] {ICMP} 192.168.56.30 -> 192.168.56.20
06/15-11:24:07.118302 [**] [1:1000003:1] Possible TCP SYN Scan [**]
   [Priority: 0] {TCP} 192.168.56.30:44321 -> 192.168.56.20:80
06/16-10:02:51.774410 [**] [1:1000002:1] Possible SSH Brute Force [**]
   [Priority: 0] {TCP} 192.168.56.30:51876 -> 192.168.56.20:22
06/17-14:15:09.390027 [**] [1:1000004:1] Possible SQL Injection Attempt [**]
   [Priority: 0] {TCP} 192.168.56.30:49722 -> 192.168.56.20:80
```

Figure 9.2: Alerts generated by Snort

9.3 Python Port Scan Detector Output

Displays the output of the Python detector when a port scan is sent to the target. An alert is printed when 15 or more distinct ports are contacted by one source within 10 seconds.

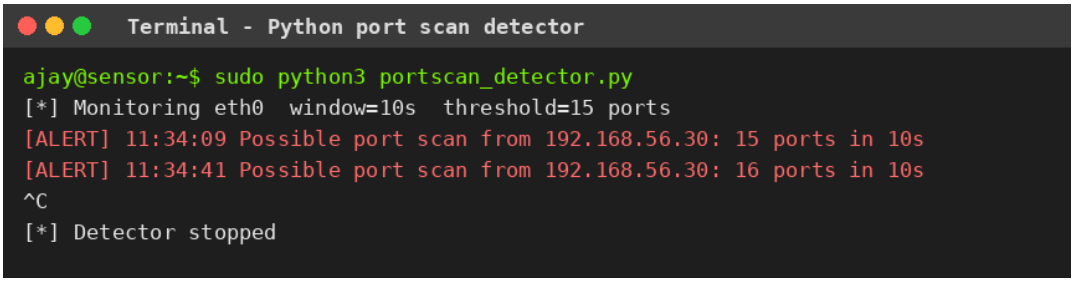


Figure 9.3: Output of the Python port scan detector

9.4 Alert Summary Output

Displays the output of the log summary script, which counts the alerts for each rule and shows the most active source addresses.

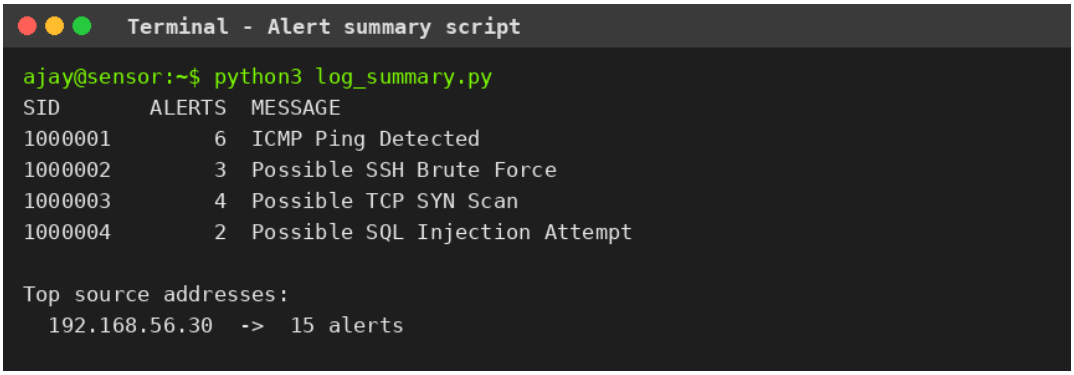


Figure 9.4: Alert summary produced by the log analysis script

10. ANALYSIS

The Intrusion Detection System was tested stage by stage to validate both the detection logic and the overall behaviour of the system.

Detection analysis: The signature rules detected ICMP echo requests, repeated SSH connection attempts, rapid SYN packets to many ports and web requests containing SQL injection keywords. The Python detector detected port scans by counting the distinct ports contacted within a time window. Using both methods together gave better coverage than either method alone.

Functional analysis: The system raised an alert only when the configured conditions were met. Normal activities such as browsing the web page and a single SSH login did not generate alerts after the thresholds were tuned, which shows that the detection_filter and the threshold logic reduce false positives.

Comparative analysis: Compared with the existing firewall-only protection, the proposed system gives visibility into allowed traffic, records alerts with the time, source and destination, and allows the administrator to find and respond to reconnaissance and password guessing at an early stage.

Test Cases

No.	Test Case	Expected Result	Status
1	Validate the Snort configuration with the test option	Configuration validated successfully	Pass
2	Ping the target from the test machine	Alert “ICMP Ping Detected”	Pass
3	Scan ports of the target using Nmap (SYN scan)	Alert “Possible TCP SYN Scan” and Python port scan alert	Pass
4	Send repeated SSH connection attempts	Alert “Possible SSH Brute Force”	Pass
5	Send a web request containing UNION SELECT	Alert “Possible SQL Injection Attempt”	Pass
6	Normal web browsing and a single SSH login	No alert (checks false positives)	Pass
7	Run the log summary script	Alerts counted by rule and source address	Pass

10.1 Challenges Faced

- **Promiscuous mode:** The sensor first did not receive the traffic of the other machines because promiscuous mode was disabled in the virtual network adapter. Enabling it in VirtualBox solved the problem.
- **False positives:** The first threshold values raised alerts for normal activity. The thresholds and time windows were tuned after testing normal traffic.
- **Rule syntax errors:** Small mistakes in the rule options prevented Snort from starting. The configuration test option (-T) was used to find and correct them.
- **Permission errors:** Packet capture requires administrator privileges, so the sensor and the Python script were run with sudo.
- **Encrypted traffic:** Content-based rules cannot read encrypted traffic, so the web test was done over plain HTTP.
- **Evasion:** Slow scans can stay below the thresholds, which showed the need for longer time windows and log correlation.

Overall, the analysis confirms that the system meets its objectives of detecting common attacks, recording clear alerts and giving the administrator better visibility of the network.

11. CONCLUSION

The **Intrusion Detection System** successfully demonstrates how open-source tools can be used to monitor a network and detect common attacks. By combining signature-based rules written for Snort with a threshold-based detector written in Python, the system detected ICMP ping sweeps, TCP SYN scans, SSH brute-force attempts and SQL injection attempts in the laboratory environment, and recorded each event as a clear alert.

The project's layered design, consisting of the traffic layer, the detection layer and the analysis layer, provides a clear separation of tasks. The project also showed that detection is not a one-time setup: it needs careful sensor placement, continuous tuning of rules and prompt analysis of alerts. All testing was carried out only on an isolated virtual network, following the principle of safe and ethical security testing.

The internship also provided valuable practical experience in Linux administration, network traffic analysis, rule writing and Python scripting. The developed system can serve as a foundation for a more advanced monitoring solution in the future.

Learning Outcomes

The internship helped me gain the following knowledge and skills:

- Understanding of the architecture, types and detection methods of intrusion detection systems.
- Hands-on experience in installing, configuring and testing Snort in a virtual laboratory.
- Skill in writing and tuning Snort rules for common attacks.
- Ability to capture and read network traffic using Wireshark and tcpdump.
- Skill in developing a detection script and a log analysis script in Python using Scapy.
- Knowledge of evaluation metrics such as precision, recall and false positive rate.
- Awareness of the limitations of IDS such as encrypted traffic, evasion and false positives.
- Understanding of ethical and legal responsibilities in security testing.

12. FUTURE WORK

- Integrating **machine learning models** trained on public datasets such as NSL-KDD or CIC-IDS2017 to detect unknown attacks.
- Forwarding the alerts to a **SIEM** such as the Elastic Stack or Splunk for dashboards and correlation.
- Developing a **web dashboard** to display live alerts, charts and top source addresses.
- Adding **automatic response**, such as temporarily blocking an attacking address on the firewall.
- Adding a **host-based IDS** such as Wazuh to create a hybrid system.
- Using **threat intelligence feeds** of known malicious addresses to enrich and prioritise alerts.
- Adding detection for more attacks such as ARP spoofing, DNS tunnelling and SYN floods.

- Sending **email or messaging notifications** to the administrator for high-priority alerts.
- Studying **TLS inspection** and metadata analysis to monitor encrypted traffic.

13. REFERENCES

1. **NIST Special Publication 800-94** – Guide to Intrusion Detection and Prevention Systems (IDPS), Scarfone, K. and Mell, P.
2. **Snort Documentation** – Snort users manual and rule writing guide. https://www.snort.org
3. **Suricata Documentation** – Suricata user guide. https://suricata.io
4. **Zeek Documentation** – Network security monitoring framework. https://zeek.org
5. **Scapy Documentation** – Packet manipulation library for Python. https://scapy.net
6. **Wireshark Documentation** – Network protocol analyser. https://www.wireshark.org
7. **Nmap Reference Guide** – Network scanning tool. https://nmap.org
8. **MITRE ATT&CK** – Knowledge base of adversary tactics and techniques. https://attack.mitre.org
9. **Denning, D. E.** (1987). "An Intrusion-Detection Model." IEEE Transactions on Software Engineering.
10. **OWASP Top Ten** – Web application security risks. https://owasp.org/www-project-top-ten/